
Functional PCA for Dimensionality Reduction

(Machine Learning 2025 Course)

Anita Toleutaeva¹ Denis Suchkov¹ Ildar Saiapov¹ Oleg Kobzarev¹

Abstract

Dimensionality reduction is a fundamental tool in data analysis. The most widely used method is Principal Component Analysis (PCA), which is a linear technique. However, there are many applications where data cannot be analyzed as a set of features, but rather as functions evaluated at a grid. Functional Principal Component Analysis (FPCA) is a method that extends PCA to these types of data. It extracts smooth basis functions that capture the structure of functional data.

In this work, we review the theory and implementation of FPCA and discuss its applications in feature extraction. We present experiments on synthetic data, MNIST digits, and multi-resolution images. The results show that FPCA can extract more interpretable patterns than classical PCA from functional data.

Github repo: github.com/ML_FPCA

Presentation file: github.com/ML_FPCA_pres

1. Introduction

Functional data, represented as continuous curves or functions, appears in applications like biomedical signals, spectroscopy, and finance. Classical Principal Component Analysis (PCA) reduces dimensionality in multivariate data, but it struggles with functional data due to its infinite-dimensional, smooth nature. Functional Principal Component Analysis (FPCA) addresses this by decomposing functions into a series of orthogonal basis functions (eigenfunctions), capturing key variations.

¹Skolkovo Institute of Science and Technology, Moscow, Russia. Correspondence to: Anita Toleutaeva <Anita.Toleutaeva@skoltech.ru>, Denis Suchkov <Denis.Suchkov@skoltech.ru>, Ildar Saiapov <Ildar.Saiapov@skoltech.ru>, Oleg Kobzarev <Oleg.Kobzarev@skoltech.ru>.

The aim of this project is to conduct a literature review on FPCA and its current algorithms and implementations. We will also evaluate its applications in feature extraction and discuss its limitations, along with strategies for overcoming them. Additionally, we will compare FPCA to PCA and FCPTA and present experimental results on several datasets.

2. Preliminaries

In this section, we briefly outline the notation and key concepts underlying PCA and its extension to functional data.

Let $x_i \in \mathbf{R}^p$ for $i = \overline{1, n}$ be a dataset of n observations and p number of features in the classical (multivariate) setting. The data are then centered by subtracting μ and their covariance matrix is given by

$$\Sigma = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)(x_i - \mu)^\top.$$

PCA is defined as an orthogonal linear transformation on a real inner product space that transforms the data $X = (x_1, \dots, x_n)^\top \in \mathbf{R}^{n \times p}$ with column-wise zero empirical mean $\mu^{(j)} = \frac{1}{n} \sum_{i=1}^n x_i^j = 0$, $j = \overline{1, p}$, to a new coordinate system with basis being a set of eigenvectors $\{v_k\}_{k=1}^p$:

$$\Sigma v_k = \lambda_k v_k, \quad \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0.$$

The greatest variance by some scalar projection of the data comes to lie on the first coordinate (called the first principal component PC), the second greatest variance on the second coordinate, and so on (Jolliffe, 2002): $z_{ik} = (x_i - \mu)^\top v_k$.

FPCA extends classical PCA to data observed as functions. For functional data, we model each observation as a function $X_i(t)$ defined on a continuous domain $\mathcal{T}$, then decompose it via the Karhunen–Loeve expansion:

$$X_i(t) = \mu(t) + \sum_{k=1}^{\infty} \xi_{ik} \varphi_k(t),$$

where $\mu(t)$ is the mean function, $\{\varphi_k(t)\}$ are orthonormal eigenfunctions obtained from the covariance operator $G(s, t)$ instead of a finite-dimensional covariance matrix Σ :

$$G(s, t) = \text{Cov}(X(s), X(t)) =$$

$$= \mathbb{E}[(X(s) - \mu(s))(X(t) - \mu(t))].$$

Eigenfunctions and eigenvalues satisfy the integral equation:

$$\int_{\mathcal{T}} G(s, t) \varphi_k(s) ds = \lambda_k \varphi_k(t), \quad t \in \mathcal{T}.$$

ξ_{ik} are the functional principal component scores:

$$\begin{aligned} \xi_{ik} &= \int_{\mathcal{T}} (X_i(t) - \mu(t)) \varphi_k(t) dt \approx \\ &\approx \sum_{j=1}^M (X_i(t_j) - \mu(t_j)) \varphi_k(t_j) \Delta t_j, \end{aligned}$$

This representation reduces the infinite-dimensional problem to a finite one by truncating the series after M terms. The eigenfunctions are chosen to maximize the explained variance in the L^2 norm, similar to how classical PCA selects eigenvectors from the covariance matrix.

3. Literature Review

Our review covers key contributions to FPCA, including theory, computation, and applications. Studies compare discretization, basis expansion, and numerical methods, propose solutions like PACE for sparse data, and extend FPCA to multivariate settings. However, challenges remain, such as numerical stability and irregular sampling, driving further research.

Classical PCA has been widely studied for its role in dimensionality reduction in multivariate data (Jolliffe, 2002). FPCA, as introduced in (Karhunen, 1946) and developed in (Yao et al., 2005), adapts these ideas to settings where observations are curves or functions, providing natural representation for continuous data like time series.

(Wang et al., 2016) provides a theoretical foundation for FPCA and examine alternative approaches (to mitigate sparse, noisy data and other issues): ranging from mixed-effects modeling and kernel-based smoothing to the aforementioned PACE method. Authors establish convergence rates, for example,

$$\sup_{t \in \mathcal{T}} |\hat{\mu}(t) - \mu(t)| = O_p \left(\frac{1}{\sqrt{nh}} \right),$$

and similar rates for smoothed $\hat{G}(s, t)$ and the eigenfunctions. They also emphasize that FPCA is central for dimension reduction and feature extraction in functional data, and inform current research efforts aimed at refining FPCA methodologies.

A Survey of FPCA (Shang, 2013) categorizes computational approaches into three main types: First, a discretization,

which renormalizes and interpolates eigenvectors on a dense grid. Second, the basis expansion, representing $X_i(t)$ as a sum of fixed basis functions: $\{b_j(t)\}$:

$$X_i(t) \approx \sum_{j=1}^M c_{ij} b_j(t)$$

Third, numerical approximation, using quadrature rules for irregular data. The survey also covers FPCA extensions, including smoothed, robust, sparse, common, and multilevel variants.

In a similar vein (Hall et al., 2006) authors address the critical issue of sparsity in longitudinal data through their PACE (Principal Components Analysis through Conditional Expectation) method. Their pipeline includes:

1. Estimating the mean $\hat{\mu}(t)$ and covariance $\hat{G}(s, t)$ via local smoothing;
2. Computing eigenfunctions $\hat{\phi}_k(t)$ and eigenvalues $\hat{\lambda}_k$ from $\hat{G}(s, t)$;
3. Obtaining FPC scores using conditional expectation rather than standard numerical integration:

$$\hat{\xi}_{ik} = \hat{\lambda}_k \hat{\phi}_k^T \hat{\Sigma}_{Y_i}^{-1} (Y_i - \hat{\mu}), \quad \hat{\Sigma}_{Y_i} = \hat{G} + \hat{\sigma}^2 I.$$

This method yields consistent estimates and enables confidence band construction.

Happ and Greven (2018) (Happ & Greven, 2018) extend FPCA to multivariate functional data across heterogeneous domains (e.g., curves vs. images). Using the Karhunen–Loève theorem, their method balances variation via weighted decomposition, producing multivariate eigenfunctions $\{\phi_k^{(m)}(t)\}$ and scores—an alternative to tensor PCA.

Recent studies address algorithmic improvements and practical challenges. The scikit-fda package (Berrendero et al., 2020) implements FPCA with basis expansion, mean/covariance estimation, and eigenanalysis. However, it faces numerical instability and parameter sensitivity, particularly with irregular or sparse data—issues we explore in detail.

4. Experimental Setup and Results

We chose the FDAPy library for this study because it is actively maintained and offers a robust implementation of FPCA, compared to alternatives like scikit-fda.

4.1. Toy Problem on Noisy Polynomial Functions

Objective: To test FPCA's ability to capture the underlying structure of noisy polynomial functions and predict the polynomial order.

4.1.1. SETUP

- **Data Generation:** We synthetically generated 60 functions using an 8th-degree polynomial with coefficients drawn from a normal distribution (scaled by a margin of 3). Each function is evaluated on a dense grid of 100 points spanning $[-3, 3]$ (domain centered at zero) and $[2, 4]$ (with shifted domain).
- **Noise Addition:** Gaussian noise (with high variance, e.g. std 1000).

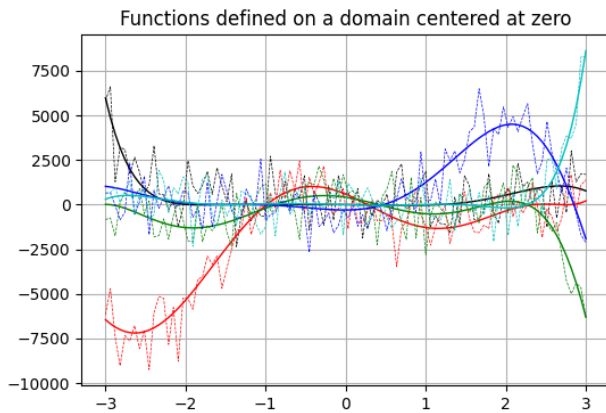


Figure 1. Example of Polynomial Functions: true and observed data. Dashed line corresponds to noisy data.

- **Preprocessing:** The functions were converted to the FDAPy “DenseFunctionalData” format by assigning evenly spaced argument values.

4.1.2. EXPERIMENTS

We applied FPCA using the covariance-based method provided by FDAPy. FPCA scores were computed using the PACE method, and, finally, the FPCA model is used to transform the functional data to a lower-dimensional feature space, and then an inverse transformation reconstructs the original image.

FPCA using 3 components. FPCA with 3 components showed that this was insufficient to capture the full function shape. However, when the domain shifts away from zero, a dominant polynomial term emerges, allowing even 3 components to capture most essential behavior. Additional components had little impact as the leading term dominated the signal.

FPCA using 8 components (matching the polynomial order). The eigenfunctions were much closer in shape to the true functions, and the FPCA scores indicated a better representation of the data.

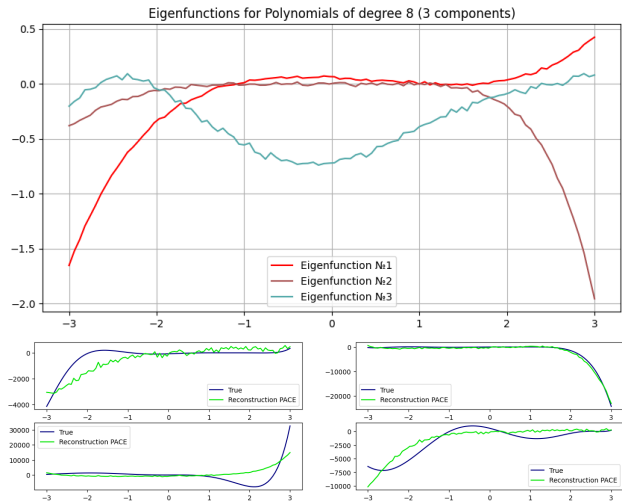


Figure 2. Reconstruction showed that the basis of 3 PC was too limited to accurately restore the original functions.

Adaptive Number of Components: Automatically selecting the number of components to explain a given proportion (e.g., 99%) of the total variance.

When the domain is centered at zero, the method selected 7 components, which is close to the true number (8). When the domain is shifted, the method selected just 1 component, extra components beyond the dominant one added little new information (mostly noise).

In the case with high noise level, the adaptive method selected an excessive number of components (e.g., 51 components), far exceeding the true polynomial order and modeling mostly noise rather than the underlying function structure.

4.1.3. KEY OBSERVATIONS

1. Complex functions (e.g., near zero where multiple polynomial terms interact) require more components to capture the signal, while simpler domains where one term dominates may need fewer.
2. In scenarios with **high noise**, **all methods struggle to recover the underlying function**. Adaptive selection may overestimate the number of components by capturing noise variance.
3. A proper balance between the number of components and the noise level is crucial. For well-posed problems where the signal is clearly defined, **adaptive methods can approximate the true underlying structure**. In cases where noise dominates, additional preprocessing or noise reduction techniques are needed before applying FPCA.

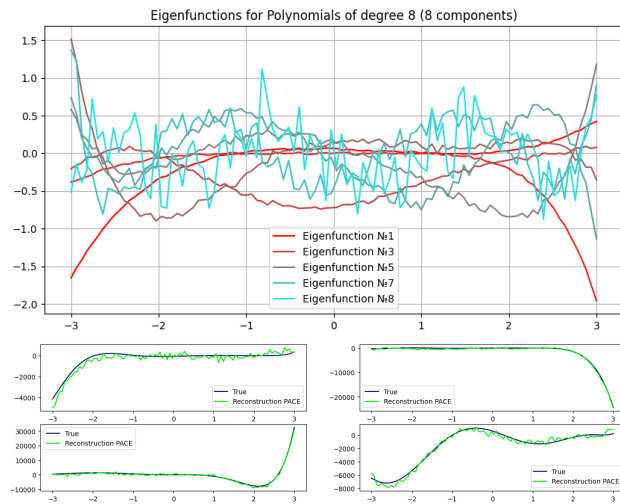


Figure 3. The reconstruction with 8 components improved relative to the 3-component case, although the inherent noise still prevented perfect recovery.

4.2. FPCA on the MNIST Dataset for Digit Classification

Objective: To explore how well FPCA could capture the underlying structure of the handwritten digit images. We compared different fixed numbers of components (3, 10, 30) as well as an adaptive method to assess their impact on downstream classification performance.

4.2.1. SETUP

- **Dataset:** MNIST – 70,000 grayscale images (28×28 pixels) downloaded from Keras. This dataset already has a train test split, we used it.
- **Preprocessing:** Images were normalized by scaling pixel values to [0, 1]. Each image was flattened and then converted into a functional data object using a dense, evenly spaced grid over the interval [0, 1].
- **Transformation:** The DenseFunctionalData representation from FDAPy.

Experiment with Fixed Components: FPCA was applied with 3, 10, and 30 components. For each setting, the eigenfunctions were visualized (after reshaping to the original 2D format), and the FPCA scores were used as features.

Experiment with Adaptive Component Selection: An adaptive approach was attempted where FPCA was set to explain 85% of the variance. However, due to memory limitations, the method selected an excessive number of components. This indicates that in practice the optimal

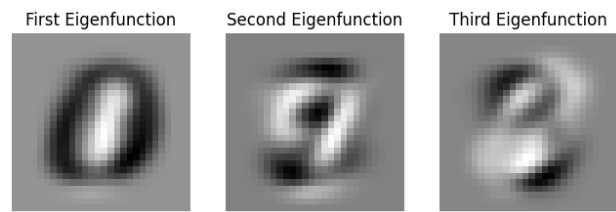


Figure 4. The first eigenfunctions are the same for decomposition with any number of components.

number is better determined by empirical tuning rather than an automatic variance threshold.

Classification: The FPCA scores were fed into a standard SVM classifier (with a linear kernel) to evaluate the quality of the feature extraction.

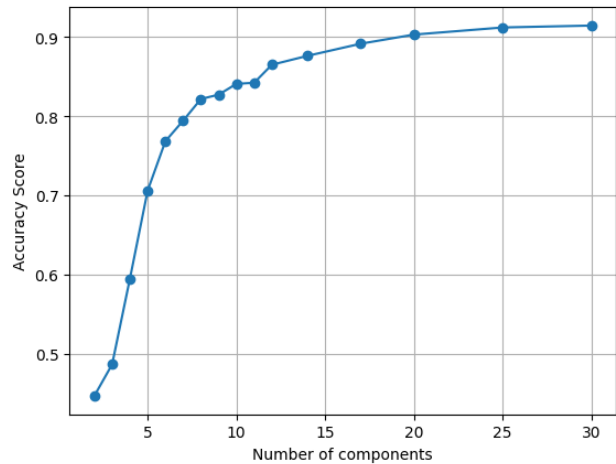


Figure 5. With 3 components, classification accuracy was around 48.7%. With 10 components, the accuracy improved dramatically to approximately 84.1%. Further improvements are slowing down, reaching 91.5% accuracy with 30 components

4.2.2. KEY OBSERVATIONS

1. A low number of components (e.g. 3) does not capture enough structural detail for effective classification.
2. As the number of components increases, both the reconstructed images and classification accuracy improve, indicating that the key functional modes are better captured. However, an aggressive adaptive selection (above 85% explained variance) may lead to **memory issues** and **overfitting of noise**.
3. The optimal range for this dataset is 20–30 components.

4.3. Multi-Resolution Image Experiments (CIFAR-10)

Objective: This experiment aims to validate that FPCA can extract meaningful modes of image variation independent of resolution. By applying FPCA on grayscale CIFAR-10 images resized to grids from 24×24 to 96×96, we evaluate the interpretability of principal components, consistency across discretizations, and the impact on reconstruction quality and classification performance.

4.3.1. SETUP

- **Dataset:** CIFAR-10, converted to grayscale. This dataset already has a train test split, we used it.
- **Preprocessing:** Create different resolution sets by re-sizing images to 24×24, 32×32, 48×48, 64×64, and 96×96 using a zoom function. Each set is vectorized and then converted to a DenseFunctionalData.

The eigenfunctions are visualized and compared across resolutions. This experiment shows that FPCA can work with different sizes of grids, learning same global and local patterns similarly.

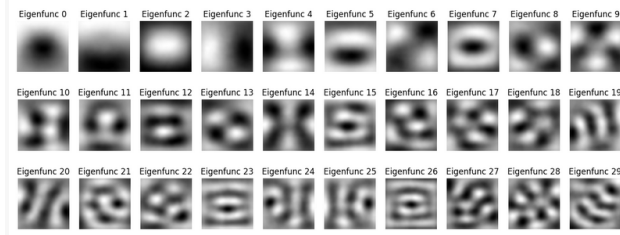


Figure 6. The first 10 eigenfunctions capture global intensity gradients, while additional components refine local details.

CatBoost is employed as classifier due to its ability to model non-linear relationships, and XGBoost, MLPClassifier were also tested as alternatives. We use the weighted F1-score to assess classification performance, which is particularly useful in the presence of class imbalances.

The first 10–15 components capture fundamental modes across resolutions, while 16–30 refine local details with minimal impact on classification performance. At 30 components, FPCA captures nuanced features, but higher-order eigenfunctions become specialized, with accuracy saturating before 30.

We used reconstruction metrics (MSE, PSNR, SSIM) to evaluate the quality of recovered images.

PSNR (Peak Signal-to-Noise Ratio) - commonly used to quantify reconstruction quality for images and video subject to lossy compression.

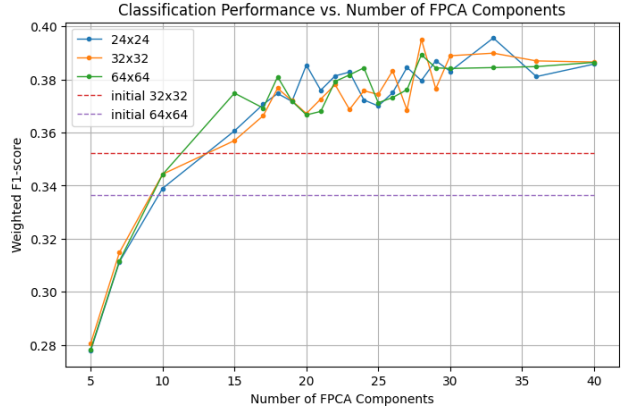


Figure 7. Most FPCA-based curves approach or exceed this baseline once they pass about 10–15 components, implying that FPCA can handle data with different grid sizes. Accuracy with FPCA components via CatBoost exceeds accuracy of classification of the initial data.

Table 1. Reconstruction Results.

NCOMP	PSNR _{32×32}	SSIM _{32×32}	MSE _{32×32}
10	17.4069	0.3708	0.0182
20	18.8300	0.4404	0.0131
30	19.7742	0.4883	0.0105
40	20.5072	0.5303	0.0089

$$\text{PSNR}(X, \hat{X}) = 10 \ln \left(\frac{\max(X)^2}{\text{MSE}(X, \hat{X})} \right)$$

SSIM (Structural Similarity Index) - perceptual metric that quantifies image quality degradation caused by processing, such as data compression.

$$\text{SSIM}(X, \hat{X}) = \frac{(2\mu_X\mu_{\hat{X}} + C_1)(2\sigma_{X\hat{X}} + C_2)}{(\mu_X^2 + \mu_{\hat{X}}^2 + C_1)(\sigma_X^2 + \sigma_{\hat{X}}^2 + C_2)}$$

Where μ_X and σ_X^2 - mean and std, $\sigma_{X\hat{X}}$ - covariation, C_1 and C_2 - constants. Better reflects visual similarity, considers brightness, contrast, and structure.

From table and picture above, we can conclude that reconstruction quality continues to grow stably. Each following eigenfunction represents more sophisticated features, but they don't play much role in the classification task.

4.3.2. KEY OBSERVATIONS

1. With too few components (e.g., 5), the F1-score is low (0.27 score). 10–15 components capture 70–80% of meaningful variance (0.4 score). The optimal number

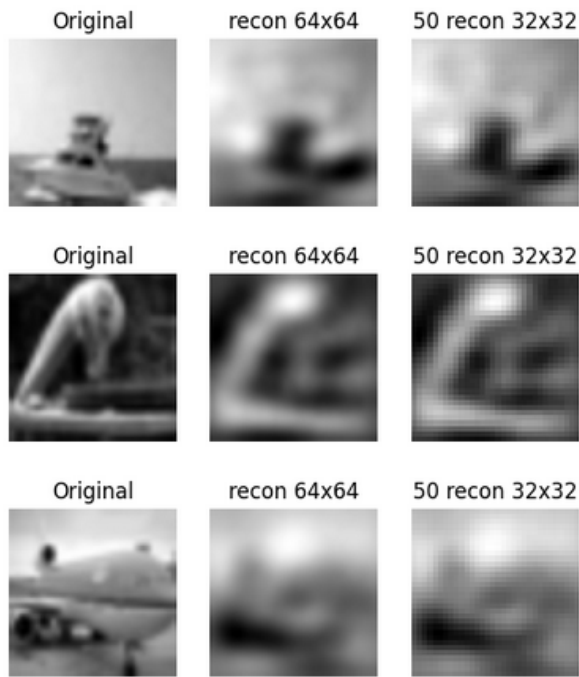


Figure 8. An illustration of image FPCA reconstruction results for different resolutions on CIFAR10.

of components is in the range of 20-30: High-order eigenvalues capture complex features, but a further increase in basis may lead to overfitting.

2. Reconstruction metrics indicate that adding more components reduces MSE and increases PSNR/SSIM until a plateau is reached.
3. While lower resolutions yield smoother, simpler eigenfunctions, higher resolutions capture additional fine details at the expense of increased noise and computational cost.

4.4. Experiments on the HaGRID Dataset

The goal is to determine how many components are sufficient to capture the meaningful variance in high-dimensional image data while preserving the structural features required for accurate classification.

4.4.1. SETUP

- **Dataset:** The HaGRID dataset consists of 1086158 RGB hand gestures images and annotations, we used subset which comprises over 10,600 training images and 2,200 test images with labels from several gesture classes (e.g., call, dislike, fist, peace, etc.).

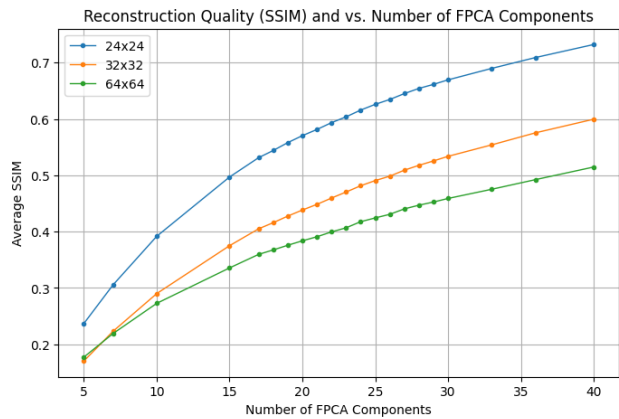


Figure 9. An illustration that increasing number of components only helps restore initial images, because accuracy increases even on the big values of components (30 and more), when classification accuracy reaches plateau (near 0.4 f1 score).

- **Preprocessing:** We load images and extract bounding box information from JSON annotations, crop images according to this information, and resize each cropped region to a fixed size (64×64). Each set is vectorized and then converted to a DenseFunctionalData.

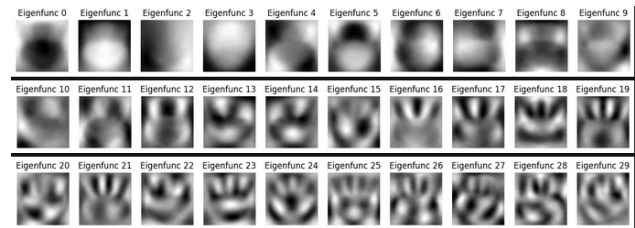


Figure 10. The FPCA eigenfunctions extracted from the HaGRID dataset starting from the 15th demonstrate quite familiar finger patterns.

Here we also used CatBoost classifier to model non-linearity and F1-score to evaluate classification performance on imbalanced data.

We evaluated the quality of reconstruction of images from FPCA-transformed features using metrics such as the mean squared error (MSE), the peak signal-to-noise ratio (PSNR) and the structural similarity index (SSIM).

4.4.2. KEY OBSERVATIONS

1. FPCA not only reduces dimensionality, but also provides robust features for downstream tasks such as classification. The multi-resolution experiments confirm that the extracted modes are consistent across different

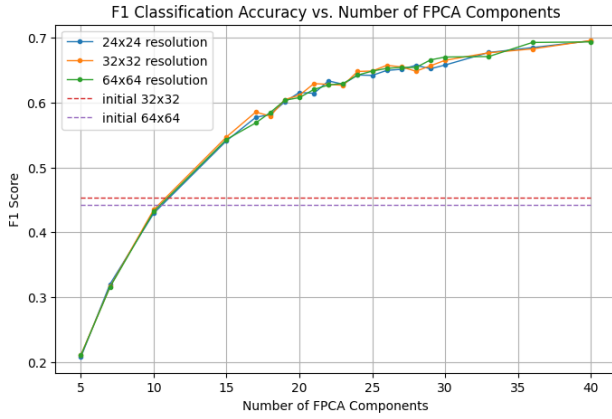


Figure 11. Scores increase from around 0.21 (5 components) to approximately 0.70 (40 components), with the steepest gains observed between 5–10 components and saturation near 20–30 components.

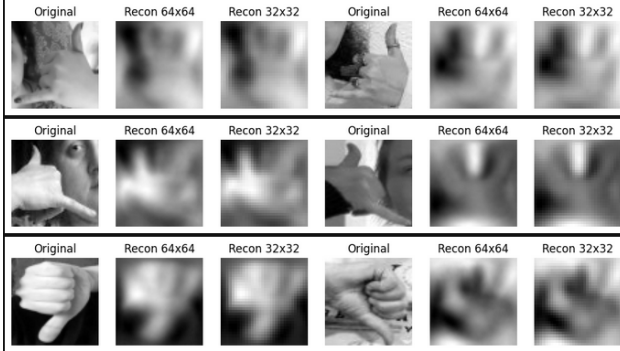


Figure 12. Reconstruction using 40 components. Comparisons reveal that global structures (shapes, major intensity gradients) are well preserved across all settings, additional components refine local details.

image sizes, though higher resolutions demand more computational resources.

5. Challenges and Reproduction of FPCA Methods in High Dimensions

The curse of dimensionality is a well-known phenomenon which refers to the exponential increase in data sparsity as the number of dimensions grows. Classical FPCA is suited for functional data with a low number of functional variables p relative to observations n . However, modern applications (e.g., neuroimaging, longitudinal studies with dropout) often have $p \gg n$ or irregular sampling, presenting two key problems due to this curse:

1. **Estimating the full covariance operator** $\text{Cov}(X(s), X(t))$ becomes unstable when p is

large. Hu and Yao (Hu & Yao, 2021) propose a sparse FPCA approach that projects functional observations onto a finite orthonormal basis (e.g., B-splines or wavelets) and performs a thresholded multivariate Karhunen–Loève expansion. This reduces the effective dimensionality and leads to a computational complexity of $O(np s_n + nN^2 + N^3)$, where N is the number of retained coefficients $N \gg p s_n$.

2. **Generalization ability of the learned basis** (eigenfunctions). Shi et al. (Shi et al., 2020) address scenarios with informative dropout by incorporating a likelihood-based framework (imFunPCA), which avoids biased covariance estimates. For non-Gaussian data, Zhong et al. (Zhong et al., 2021) replace the traditional covariance function with a Kendall’s T function, which is robust to outliers and heavy-tailed distributions with the same eigenfunctions as the covariance operator. This method achieves a convergence rate of $O(N^{-1/2})$ under minimal distributional assumptions.

Additional strategies, such as the multivariate extension of FPCA (d-FPCA) (Chen & Jiang, 2016), leverage GPU parallelization, fast Fourier transforms, and random projection to further reduce computational burden on large covariance matrices.

5.1. FCPTA on the HAM10000 Dermatoscopic Images (Multi-Resolution Image Dataset)

To evaluate these methods, we reproduced aspects of the sparse and robust FPCA approaches on a multi-resolution dataset of dermatoscopic images. The main innovation of the proposed sFPCA lies in its integration of sparsity assumptions within the FPCA framework for high-dimensional functional data.

5.1.1. SETUP

- HAM10000, available from the ISIC archive, includes images of seven skin cancer types (e.g., Melanoma, Basal cell carcinoma, Melanocytic nevi, etc.).
- Data cleaning and balancing were performed by sampling 100 examples per class to mitigate class imbalance. We used train test split from sklearn with test size 0.2.
- Images were resized (e.g., to 512×512 for initial visualization, then further downsampled to 24×24, 32×32, and 64×64 for FPCA analysis).
- Conversion to grayscale and vectorization were applied, and the images were then converted into DenseFunctionalData objects (for both 1D and 2D representations, with an evenly spaced grid in the domain).

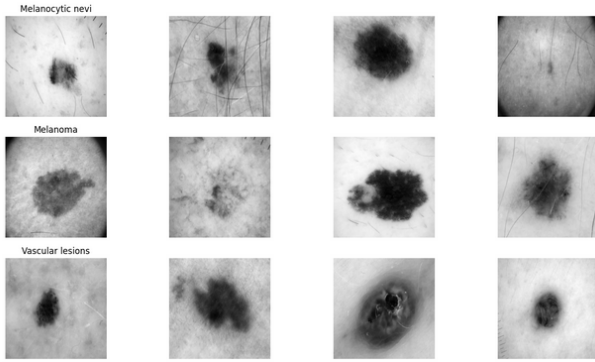


Figure 13. Example images for different classes from the HAM10000 dataset.

Experiment with FCPTA for 2D Data: To manage memory, sparse penalty matrices were created for each image dimension using a first-order difference operator, ensuring smoothness of the eigenfunctions.

We set the number of components (e.g., 50) and tuned smoothing parameters (alpha values) within a specified range.

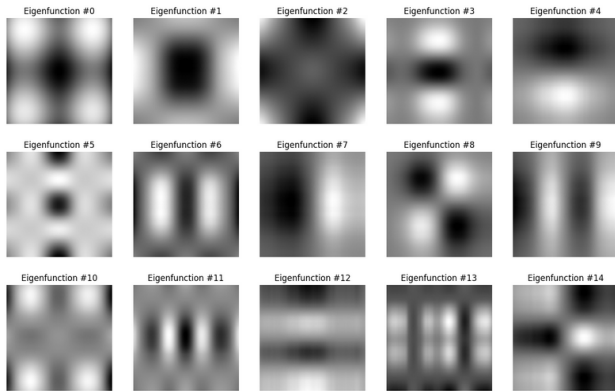


Figure 14. The FCPTA eigenfunctions exhibit more linear or banded patterns, suggesting sensitivity to directional features (e.g., horizontal/vertical edges, illumination artifacts).

Experiment with FPCA for 1D Data.

Accuracy. Both methods show fluctuating accuracy as the number of PC increases; there is no smooth, monotonic improvement. In some ranges (25–50 components), FCPTA outperforms FPCA, whereas in other ranges (e.g., near 150 components), FPCA has the advantage.

5.1.2. KEY OBSERVATIONS

1. FPCA may emphasize more global, radially oriented structures, potentially useful for overall lesion shape characterization.

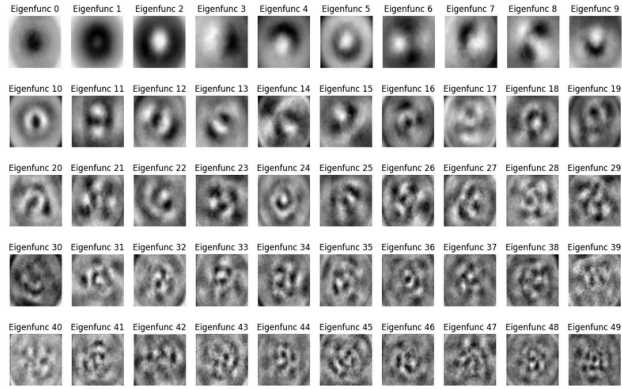
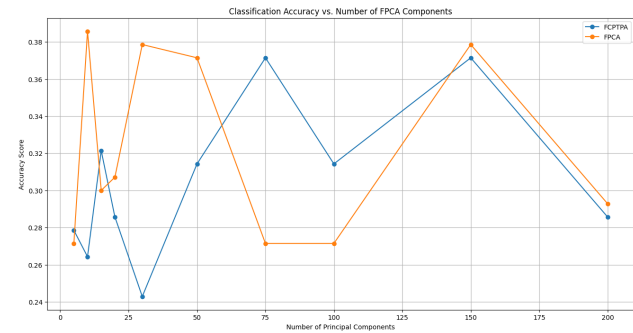


Figure 15. The FPCA eigenfunctions appear to capture more circular or radially symmetric intensity patterns, reflecting global variations (e.g., central lesion vs. boundary transitions).



2. FCPTA might better capture directional or edge-based patterns, which can be crucial for certain lesion subtypes.
3. The diverse shapes of PC highlight the dataset's complexity, therefore decomposition may not be very informative for this images.

6. Conclusion

In summary, FPCA effectively generalizes classical PCA for functional data by leveraging smooth basis functions to reduce dimensionality while preserving the continuous nature of the data. The comparative experiments indicate that FPCA offers improved interpretability and feature extraction over traditional methods when applied to varied datasets. Nonetheless, the method's performance is highly sensitive to the number of components chosen and the level of noise present, pointing to the necessity for improved adaptive selection and noise reduction techniques. Future work should focus on optimizing parameter tuning and incorporating sparsity constraints to enhance scalability and reliability in high-dimensional settings.

References

- Berrendero, J. et al. scikit-fda: A python package for functional data analysis. *Journal of Open Source Software*, 5(52):2979, 2020. URL https://www.researchgate.net/publication/380415756_scikit-fda_A_Python_Package_for_Functional_Data_Analysis.
- Chen, L.-H. and Jiang, C.-R. Multi-dimensional functional principal component analysis. *Preprint*, Mar 2016. Extends FPCA to multivariate (d-dimensional) functional data.
- Hall, P., Müller, H. G., and Wang, J. L. Properties of principal component methods for functional and longitudinal data analysis. *Annals of Statistics*, 34(1):1493–1517, 2006. doi: 10.1214/009053606000001090.
- Happ, C. and Greven, S. Multivariate functional principal component analysiswang2016functional for data observed on different (dimensional) domains. *Journal of the American Statistical Association*, 2018. Volume, number, and page details to be added if available.
- Hu, X. and Yao, F. Sparse functional principal component analysis in high dimensions. *Preprint*, 2021. Available online.
- Jolliffe, I. T. Principal component analysis. springer series in statistics. *Annual Review of Statistics and Its Application*, 2002. doi: doi:10.1007/b98835. URL <https://link.springer.com/book/10.1007/b98835>.
- Karhunen, K. Zur spektraltheorie stochastis r prozesse. *Annales Academiae Scientiarum Fennicae, Series A. I. Mathematica-Physica*, 37:1–79, 1946.
- Shang, H. L. A survey of functional principal component analysis. *AStA Advances in Statistical Analysis*, 98(2): 121–142, 2013. doi: 10.1007/s10182-013-0213-1.
- Shi, H., Dong, J., Wang, L., and Cao, J. Functional principal component analysis for longitudinal data with informative dropout. *Preprint*, 2020. Incorporates dropout information into FPCA estimation.
- Ullah, S., F. C. Applications of functional data analysis: A systematic review. *BMC Medical Research Methodology*, 13, 2013. doi: 10.1214/009053606000001090. URL <https://doi.org/10.1186/1471-2288-13-43>.
- Wang, J.-L., Chiou, J.-M., and Müller, H.-G. Functional data analysis. *Annual Review of Statistics and Its Application*, 3:257–295, 2016. doi: 10.1146/annurev-statistics-041715-033624.
- Yao, F., Müller, H.-G., and Wang, J.-L. Functional data analysis for sparse longitudinal data. *Journal of the American Statistical Association*, 2005. URL <https://anson.ucdavis.edu/~mueller/jasa03-190final.pdf>.
- Zhang Y, Zhou J, N. F. D. J. H. R. P. W. e. a. Characterizing early child growth patterns of height-for-age in an urban slum cohort of bangladesh with functional principal component analysis. *BMC Medical Research Methodology*, 2017.
- Zhong, R., Liu, S., Li, H., and Zhang, J. Functional principal component analysis estimator for non-gaussian data. *Preprint*, 2021. Robust FPCA via a Kendall’s T function is proposed.

A. Comparison of PCA and FPCA

Classical PCA is a dimensionality reduction method based on finding an orthonormal set of vectors (eigenvectors of the covariance matrix) that explain the most variance in the data, while FPCA is a generalization of PCA to functional data, that is, to cases where each observed point is a function observed on some continuous set (e.g., a curve in time)

In classical PCA, data should be represented as a matrix where each object has the same number of features. However, in real-world tasks, data are often measured unevenly:

1. Time series with different measurement frequencies.
2. Images with different resolutions.
3. Signals with missing values.

FPCA can handle data with different resolutions: it converts data into functional form, ignoring the original measurement grid. This allows you to analyze data that cannot be aligned (e.g. time series with gaps). FPCA retains information about smoothness and correlation between neighboring points. FPCA is learning global and local intensity patterns. This underscores how FPCA progressively captures different scales of variation.

The practical advantages of FPCA are demonstrated in a variety of applications. In the study of child growth by Zhang et al. (Zhang Y, 2017), FPCA was used to characterize early growth patterns. The authors show that the first functional component reflects a general deficit or excess of growth relative to the average, and the second reflects the age at which growth slows down; together, they identify differences in trajectories that are not obvious from a single average curve. Ullah and Finch (Ullah, 2013) presented a systematic review of functional data applications in various fields, from climatology to sports science, which confirms the wide applicability of FPCA to identify significant features of large collections of similar curves.

In summary, both methods pursue the same goal - dimensionality reduction with minimal loss of information - but FPCA does it by taking into account continuity and smoothness of data, while PCA does it on a purely algebraic vector level. Correct application of each method reveals maximum advantages: **FPCA - in the analysis of functional structures, PCA - in classical multivariate data analysis.**

B. Team member's contributions

Explicitly stated contributions of each team member to the final project.

Table 2. Comparability table of PCA and FPCA methods.

Criterion	PCA	FPCA
Data type	Data with a fixed number of features.	Functional data (continuous functions: time series, trajectories, images).
Data Grid	Requires the same data structure (all objects measured at the same points)	Works with uneven or sparse grid.
Dimensionality	Reduces the dimensionality of discrete features.	Reduces the dimensionality of functional data, preserving its continuous nature.
Regularization	Sensitive to noise, requires explicit regularization.	Built-in smoothing by basis functions (e.g. splines).
Interpretation	Principal components are vectors in the original space.	Principal components are functions describing directions of variability.
Computational complexity	Effective for moderate dimensional data.	Requires function approximation (e.g., splines), which can be costly.

Anita Toleutaeva (25% of work)

- Prepared the final report
- Performed analysis for CIFAR10 and HaGRID datasets
- Selected and explored image quality metrics for the datasets used

Ildar Saiapov (25% of work)

- Reviewed literature on the topic (3 papers)
- Prepared a synthesis matrix of key papers
- Prepared a presentation

Denis Suchkov (25% of work)

- Performed analysis for toy problem and MNIST datasets
- Compiled the final report and made an Appendix
- Prepared Github repository and readme.md

Oleg Kobzarev (25% of work)

- Performed analysis for HAM10000 datasets
- Reviewing literature on the topic (3 papers)
- Participated in the preparation of the presentation

C. Computing infrastructure

To conduct the experiments we used personal computers and kaggle colab. It should be noted that a large number of FPCA components may require significant amounts of RAM, so it was carried out on a computer with such characteristics and cannot be reproduced in the collab.

D. Links to materials

1. Link to MNIST dataset kaggle.com/datasets/hojjatk/mnist-dataset.
2. Link to CIFAR10 dataset kaggle.com/c/cifar-10.
3. Link to HaGRID dataset kaggle.com/datasets/kapitanov/hagrid.
4. Link to HAM10000 dataset kaggle.com/datasets/kmader/skin-cancer-mnist-ham10000.

E. Reproducibility checklist

Answer the questions of following reproducibility checklist. If necessary, you may leave a comment.

1. A ready code was used in this project, e.g. for replication project the code from the corresponding paper was used.

☒ Yes.
☐ No.
☐ Not applicable.

General comment: If the answer is **yes**, students must explicitly clarify to which extent (e.g. which percentage of your code did you write on your own?) and which code was used.

Students' comment: We used the FDAPy library, which implements the FPCA methods we are studying. All the main code in the ipynb notebooks was written by us from scratch. Thus, we can estimate the percentage of ready code in our work at 75.

2. A clear description of the mathematical setting, algorithm, and/or model is included in the report.

☒ Yes.
☐ No.
☐ Not applicable.

Students' comment: Preliminaries

3. A link to a downloadable source code, with specification of all dependencies, including external libraries is included in the report.

☒ Yes.
☐ No.
☐ Not applicable.

Students' comment: Github repo

4. A complete description of the data collection process, including sample size, is included in the report.

☐ Yes.
☐ No.
☒ Not applicable.

Students' comment: None

5. A link to a downloadable version of the dataset or simulation environment is included in the report.

☒ Yes.
☐ No.
☐ Not applicable.

Students' comment: Appendix D.

6. An explanation of any data that were excluded, description of any pre-processing step are included in the report.

☒ Yes.
☐ No.
☐ Not applicable.

Students' comment: None

7. An explanation of how samples were allocated for training, validation and testing is included in the report.

☒ Yes.
☐ No.
☐ Not applicable.

Students' comment: None

8. The range of hyper-parameters considered, method to select the best hyper-parameter configuration, and specification of all hyper-parameters used to generate results are included in the report.

☒ Yes.
☐ No.
☐ Not applicable.

Students' comment: The report explicitly states the number of FPCA components, which is the main hyperparameter we are investigating.

9. The exact number of evaluation runs is included.

☐ Yes.
☐ No.
☒ Not applicable.

Students' comment: None

10. A description of how experiments have been conducted is included.

☒ Yes.
☐ No.
☐ Not applicable.

Students' comment: None

11. A clear definition of the specific measure or statistics used to report results is included in the report.

☒ Yes.
☐ No.
☐ Not applicable.

Students' comment: None

12. Clearly defined error bars are included in the report.

☐ Yes.

- ☐ No.
- ☒ Not applicable.

Students' comment: None

13. A description of the computing infrastructure used is included in the report.

- ☒ Yes.
- ☐ No.
- ☐ Not applicable.

Students' comment: Appendix C.